Step 1: Simulate Concurrent Updates

Shell 1 (update -20)

```
consistency_lab> var newQuantity = product.quantity - 20;
|
| db.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity } }
| );
|
| db.products.findOne({ _id: 1 });
|
{ _id: 1, name: 'Example Product', quantity: 80, version: 1 }
```

Shell 2 (update -30)

```
consistency_lab> var newQuantity = product.quantity - 30;
|
| db.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity } }
| );
|
| db.products.findOne({ _id: 1 });
|
{ _id: 1, name: 'Example Product', quantity: 70, version: 1 }
```

Result:

```
consistency_lab> db.products.findOne({ _id: 1 });
{ _id: 1, name: 'Example Product', quantity: 70, version: 1 }
```

Observation

The final quantity is incorrect.
It should be 50 but becomes 70.

Conclusion

This shows a lost update problem, where one update overwrites another due to lack of synchronization.

Step 2: Implement Transactions

Shell 1:

```
rs0 [direct: primary] consistency_lab> sessionDB1.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity1 } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
rs0 [direct: primary] consistency_lab> printjson(sessionDB1.products.findOne({ _id: 1 }));
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 1
}
```

Shell 2:

```
rs0 [direct: primary] consistency_lab> var newQuantity2 = product2.quantity - 30;
|
| sessionDB2.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity2 } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
rs0 [direct: primary] consistency_lab> printjson(sessionDB2.products.findOne({ _id: 1 }));
{
  _id: 1,
  name: 'Example Product',
  quantity: 70,
  version: 1
}
```

An error from shell 2:

```
rs0 [direct: primary] test> var product2 = sessionDB2.products.findOne({ _id: 1 });
|
| var newQuantity2 = product2.quantity - 30;
|
| sessionDB2.products.updateOne(
|   { _id: 1 },
|   { $set: { quantity: newQuantity2 } }
| );
|
| var updated2 = sessionDB2.products.findOne({ _id: 1 });
| printjson(updated2);
|
MongoServerError[WriteConflict]: Caused by :: Write conflict during plan execution and yielding is disabled. :: Please r
etry your operation or multi-document transaction.
rs0 [direct: primary] test> session2.commitTransaction();
MongoServerError[NoSuchTransaction]: Transaction with { txnNumber: 1 } has been aborted.
```

Verify the Final Quantity Of Step 2

```
rs0 [direct: primary] consistency_lab> printjson(sessionDB1.products.findOne({ _id: 1 }));
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 1
}
```

Observation

Only one transaction is successfully committed.

Conclusion

Transactions ensure:

- Atomicity
- Consistency
- Isolation

Conflicting updates are rejected automatically.

Step 3: Implement Optimistic Locking

Shell 1:

```
rs0 [direct: primary] consistency_lab>  printjson(result);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

rs0 [direct: primary] consistency_lab> var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}
```

Shell 2:

```
rs0 [direct: primary] consistency_lab>  printjson(result);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}

rs0 [direct: primary] consistency_lab> var updatedProduct = db.products.findOne({ _id: 1 });
| printjson(updatedProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}
```

Verify the Final Quantity Of Step 3

```
rs0 [direct: primary] consistency_lab> var finalProduct = db.products.findOne({ _id: 1 });
| printjson(finalProduct);
{
  _id: 1,
  name: 'Example Product',
  quantity: 80,
  version: 2
}
```

Observation

Second update fails because the version no longer matches.

Conclusion

Optimistic locking:

- Prevents overwriting updated data
- Detects conflicts using version control
- Requires retry mechanism for failed updates